TASK 03 — API Resilience & Idempotency

Task 1 — Identify critical API dependencies

Identify all external services your API needs to complete its work.

For an order application:

Dependency	Purpose	Failure impact
Inventory Service	Checks product availability	Order may need to wait
Payment Service	Processes payment	Payment must not be charged twice
Notification Service	Sends email or SMS	Order can often succeed without notification
Database	Stores orders	Order cannot be safely confirmed if persistence fails

Step-by-step process

1. Find every service called by your Order API.
2. Decide whether each dependency is critical or optional.
3. Record its normal response time and timeout limit.
4. Decide what the application should do if it fails.
5. Document the result in your JIRA task.

Example dependency document:

Order API

├— Inventory Service: required before confirming stock

├— Payment Service: required for successful payment

├— Notification Service: retry asynchronously if appropriate

└— Order Database: required to persist the order

Expected result: A dependency list that explains what can fail and how your API will respond.

Task 2 — Configure request timeouts

A timeout limits how long your application waits for another service.

For example, if the Inventory Service is not responding, your application should not wait indefinitely.

Example configuration

For Java's HTTP client:

```
import java.net.http.HttpClient;

import java.time.Duration;

HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(2))
    .build();
```

This sets a two-second connection timeout. It does not set the full response timeout.

For a downstream request, also set a request-level timeout:

```
import java.net.URI;

import java.net.http.HttpRequest;

import java.time.Duration;

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create(
        "http://localhost:8082/api/inventory"))
    .timeout(Duration.ofSeconds(3))
    .GET()
    .build();
```

How it works

- Connection timeout: maximum time to establish a connection.
- Request timeout: maximum time allowed for the HTTP request to complete, including waiting for a response.

- Overall deadline: limits the entire operation, including retries and their delays.

The actual timeout values should depend on service latency and the API's total response-time budget.

Expected result: Your API returns a controlled error when a dependency exceeds its allowed time instead of waiting indefinitely.

Task 3 — Implement bounded retries

Retry a request when a temporary failure occurs, but stop after a limited number of attempts.

For example, the Inventory Service may temporarily return HTTP 503 because it is overloaded.

Step-by-step process

1. Send the first request.
2. If an eligible transient error occurs, wait briefly.
3. Retry the request.
4. Increase the delay between retries if using exponential backoff.
5. Stop when the maximum attempt count is reached.

Example policy:

Attempt	Action	Delay before attempt
1	Initial request	0 ms
2	First retry	200 ms
3	Second retry	400 ms

The maximum is three attempts, including the original request.

Spring Boot implementation using Resilience4j

Add a compatible Resilience4j Spring Boot dependency to your Maven project and enable the Spring AOP support required by its annotations.

import io.github.resilience4j.retry.annotation.Retry;

import org.springframework.stereotype.Service;

@Service

```
public class InventoryServiceClient {
    @Retry(name = "inventoryService")
    public String checkInventory() {
        // Replace with the real downstream HTTP call.
        return "Inventory available";
    }
}
```

Configure it in application.yml:

```
resilience4j:
  retry:
    instances:
      inventoryService:
        maxAttempts: 3
        waitDuration: 200ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
        retryExceptions:
          - java.io.IOException
```

This example retries IOException failures. Your real HTTP client may report timeouts or HTTP errors differently, so configure the exceptions and response statuses that are appropriate for that client.

Do not automatically retry every failure. A validation error will not be fixed by retrying, and a payment request must be protected against duplicate charges before it is retried.

Expected result: Temporary failures can recover without repeatedly sending requests forever.

Task 4 — Configure circuit-breaker behavior

A circuit breaker prevents your application from repeatedly calling a service that is failing.

Consider a Payment Service that takes 30 seconds to respond. Repeatedly waiting for it could exhaust your application's request-handling resources.

Three circuit-breaker states

1. CLOSED

Requests are allowed. The circuit breaker monitors failures.

2. OPEN

Calls are rejected quickly while the circuit is open. A fallback or controlled error can be returned.

3. HALF-OPEN

A limited number of test calls are permitted. If they succeed, the circuit closes; if they fail, it opens again.

Configuration example

```
resilience4j:
  circuitbreaker:
    instances:
      inventoryService:
        slidingWindowSize: 10
        minimumNumberOfCalls: 5
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
        permittedNumberOfCallsInHalfOpenState: 2
```

Example, after enough calls have been recorded, a failure rate of 50% or more can open the circuit. The minimum call count prevents the breaker from making decisions based on too few requests.

Expected result: An unhealthy service does not continuously consume resources, and the circuit breaker gives it time to recover.

Task 5 — Implement idempotency for order creation

Repeated requests for the same operation should not create repeated business effects.

Imagine the customer clicks the Place Order button, but the response does not arrive. The customer tries again.

Without idempotency, two orders might be created. With idempotency, the server recognizes the repeated operation.

Example API request

POST /api/orders

Content-Type: application/json

Idempotency-Key: order-12345

```
{
  "productName": "Laptop",
  "quantity": 1
}
```

Step-by-step process

1. The client generates a unique idempotency key for the operation.
2. The server validates the request and key.
3. The server checks whether the key has already been used.
4. If it is a new key, the server claims it safely and processes the order.
5. The server saves the order and its result.
6. If the same completed request arrives again, the server returns the original result instead of creating another order.

Example:

Request	**Expected outcome**
First request with key order-12345	Creates order 101
Repeat with the same key and payload	Returns order 101

Same key with quantity changed from 1 to 2 Returns HTTP 409 Conflict

Expected result: One logical request produces at most one order, even when the client retries.

Task 6 — Store and validate idempotency keys

Save idempotency keys in a database so that the API can recognize repeated requests, even after an application restart.

Step 1: Create a database table

A basic PostgreSQL design could be:

```
CREATE TABLE idempotency_records (
    id BIGSERIAL PRIMARY KEY,
    customer_id VARCHAR(100) NOT NULL,
    idempotency_key VARCHAR(255) NOT NULL,
    request_hash VARCHAR(64) NOT NULL,
    status VARCHAR(30) NOT NULL,
    order_id BIGINT,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
   CONSTRAINT unique_customer_key
        UNIQUE (customer_id, idempotency_key)
);
```

The unique constraint ensures that the same customer cannot claim the same key twice.

Step 2: Understand the columns

Column	Purpose
customer_id	Identifies the customer making the request
idempotency_key	Identifies the logical operation
request_hash	Detects whether the request payload changed
status	Indicates processing state, such as IN_PROGRESS or COMPLETED

order_id	References the created order
created_at	Helps manage record retention

Step 3: Validate each request

The server must check:

- Is the idempotency key present and valid?
- Does the key already exist?
- Does the stored request hash match the new payload?
- Is the original operation still processing?
- Has the original operation completed?
- Is the record still within its retention period?

If the same key is reused with different data, return 409 Conflict.

Important: A check followed by a separate insert is not enough to prevent concurrent duplicates. Use a unique database constraint and handle uniqueness conflicts correctly. Save the order and completed idempotency result atomically when they are in the same database.

Task 7 — Test duplicate and concurrent requests

Verify that your API behaves correctly when requests are repeated, sent simultaneously, or interrupted by failures.

Test A: Normal request

```
curl -X POST http://localhost:8081/api/orders \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: test-order-001" \
  -d '{"productName":"Laptop","quantity":1}'
```

Expected: The API creates one order and returns its details.

Test B: Duplicate request

Send the exact same command again with the same key.

Expected: The API returns the original order result, and the database still contains only one order for that logical operation.

Test C: Same key with different data

```
curl -X POST http://localhost:8081/api/orders \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: test-order-001" \
  -d '{"productName":"Laptop","quantity":2}'
```

Expected: HTTP 409 Conflict. The second payload must not change the original order.

Test D: Concurrent requests

Use a shell loop to send several requests simultaneously:

```
for i in $(seq 1 10); do
  curl -s -X POST http://localhost:8081/api/orders \
    -H "Content-Type: application/json" \
    -H "Idempotency-Key: concurrent-order-001" \
    -d '{"productName":"Laptop","quantity":1}' &
done
wait
```

All ten requests use the same key and payload.

Expected: Only one order is created. Depending on your implementation, requests that arrive while processing may receive an in-progress response or wait for the original result.

Test E: Downstream failure

Simulate an Inventory Service timeout or HTTP 503 response.

Expected:

- Retries stop at the configured limit.
- The circuit breaker opens when its configured conditions are met.
- The API returns a controlled failure.
- No order is incorrectly confirmed when inventory verification is required.

Task 8 — Document resilience policies and results

Record what you implemented, the reason for each setting, and the evidence that your API works correctly.

Create a document called API-Resilience-Report.md.

API Resilience & Idempotency Report

1. Objective

Protect the Order API from transient failures, slow downstream services, and duplicate requests.

2. Dependencies

- Inventory Service: checks product availability.
- Payment Service: processes payments.
- Notification Service: sends order notifications.
- Order Database: stores order details.

3. Timeout Policy

Configure connection and request timeouts for downstream calls. Example values are 2 seconds for connection establishment and 3 seconds for an individual request. Validate these values against actual service latency and the overall request deadline.

4. Retry Policy

Allow a maximum of 3 attempts for selected transient failures, using exponential backoff. Do not blindly retry non-retryable errors or operations that are not protected against duplicate effects.

5. Circuit-Breaker Policy

Use CLOSED, OPEN, and HALF-OPEN states. Configure the failure-rate threshold, minimum number of calls, open-state wait duration, and permitted half-open test calls.

6. Idempotency Policy

Require an idempotency key for order creation. Store the key, request hash, operation status, and resulting order ID. Reject reuse of a key with a different request payload.

7. Test Scenarios

- Normal order creation.

- Duplicate requests.
- Same key with different payload.
- Concurrent requests.
- Downstream timeout and HTTP 503.
- Circuit-breaker recovery.

8. Results

Record actual HTTP status codes, order counts, retry counts, response times, and circuit-breaker transitions from test execution.

9. Conclusion

The implementation is considered complete after all required tests pass and the documented behavior matches the actual API behavior.